

리눅스 시스템 프로그래밍 학습 기록

남궁명수

[2] exec() 시스템 콜의 동작 원리 분석	2
[00] 서론	2
[01] exec() 시스템 콜과 프로세스 이미지 교체	5
[01-1] exec() 호출 후 실행 흐름은 어떻게 변하며 PID 는 유지되는가?	5
[01-2] 시스템 레벨 관찰: 단일 프로세스 내 코드 영역 관찰하기	5
[01-3] 이론	7
[01-4] 추가	8
[02] fork()와 exec()의 시너지: 셸(Shell)의 동작 원리	9
[02-1] 부모 프로세스를 유지하면서 새로운 프로그램을 실행하는 방법은?	9
[02-2] 시스템 레벨 관찰: 자식 프로세스의 변신과 부모의 대기 흐름 분석	9
[02-3] 이론	10
[02-4] 추가	11
[03] execve()와 환경 변수: 보안 실행 문맥 설정	13
[03-1] 새로운 프로그램에 데이터와 환경 설정은 어떻게 전달되는가?	13
[03-2] 시스템 레벨 관찰: execve 인자(argv)와 환경변수(envp) 전달 과정	13
[03-3] 이론	14
[03-4] 추가	15
[04] 자원 상속의 그림자: exec() 이후 파일 디스크립터(FD) 생존	16
[04-1] 프로세스 이미지가 바뀌어도 부모가 열어둔 파일 권한이 상속되는가?	16
[04-2] 시스템 레벨 관찰: exec() 이후에도 닫히지 않은 FD 를 통한 무단 접근 증명 ...	16
[04-3] 이론	17
[04-4] 추가	18

[2] exec() 시스템 콜의 동작 원리 분석

[00] 서론

운영체제에서 프로세스는 단순히 프로그램을 실행하는 개념을 넘어, 실행 중인 프로그램의 상태와 자원을 포함하는 독립적인 실행 단위이다. 이전 학습에서는 `fork()` 시스템 콜을 통해 기존 프로세스의 실행 문맥이 복제되며, 부모와 자식 프로세스가 독립적인 실행 흐름을 형성하는 과정을 확인하였다. 그러나 실제 운영체제에서는 단순히 프로세스를 복제하는 것만으로 프로그램 실행이 완성되지 않는다.

새로운 프로그램을 실행하기 위해서는 기존 프로세스의 메모리 공간과 코드 영역을 제거하고, 새로운 실행 파일의 코드와 데이터로 교체하는 과정이 필요하다. 리눅스에서는 이러한 역할을 `exec()` 계열 시스템 콜이 담당한다.

`exec()`는 새로운 프로세스를 생성하는 함수가 아니다. 기존 프로세스의 PID 를 유지한 상태에서 현재 프로세스의 메모리 공간을 새로운 프로그램으로 완전히 덮어쓰는 방식으로 동작한다. 즉, 프로세스의 “신원(PID)”은 유지되지만, 내부에서 실행되는 프로그램 자체는 완전히 변경된다.

이러한 구조는 운영체제가 프로세스를 생성(`fork`)과 프로그램 실행(`exec`)이라는 두 단계로 분리하여 관리하고 있음을 보여준다. 실제로 리눅스 셸(Bash)은 새로운 명령어를 실행할 때 일반적으로 다음과 같은 순서로 동작한다.

`fork() -> exec()`

먼저, `fork()`를 통해 자식 프로세스를 생성한 뒤, 자식 프로세스 내부에서 `exec()`를 호출하여 새로운 프로그램으로 교체하는 방식이다.

이번 학습에서 `exec()` 시스템 콜이 호출될 때 기존 프로세스의 메모리 공간이 어떻게 변경되는지, PID 는 왜 유지되는지, 그리고 커널이 새로운 실행 파일을 어떤 방식으로 적재하는지를 분석한다. 또한 `strace` 를 이용하여 실제 시스템 콜 흐름을 추적함으로써 사용자 수준 함수 호출이 커널 내부 동작과 어떻게 연결되는지를 확인한다.

- `execl(path, arg0, ...)`: 인자를 목록 형태로 나열하여 전달하며, 파일의 전체 경로(Path)를 직접 지정한다.
- `execv(path, argv[])`: 인자를 배열 형태로 전달하며, 파일의 전체 경로를 지정한다.
- `execlp(file, arg0, ...)`: 인자를 목록 형태로 전달하며, PATH 환경 변수에서 파일을 검색한다.
- `execvp(file, argv[])`: 인자를 배열 형태로 전달하며, PATH 환경 변수에서 파일을 검색한다.
- `execle(path, arg0, ..., envp[])`: 인자를 목록 형태로 전달하고, 새로운 환경 변수 배열을 지정한다.
- `exeve(path, argv[], envp[])`: 인자를 배열 형태로 전달하고, 새로운 환경 변수 배열을 지정한다.

l : list 방식(인자를 하나씩 나열)

v : vector 방식(배열로 전달)

p: PATH 에서 파일 검색

e: 환경변수 직접 지정

NULL: 인자의 끝 표시

```
execl("/bin/ls", "ls", "-l", NULL);
```

- 인자를 직접 하나씩 적음
- 경로 직접 지정

```
char *argv[] = {"ls", "-l", NULL};
execv("/bin/ls", argv);
```

- 배열로 전달
- 경로 직접 지정

```
execlp("ls", "ls", "-l", NULL);
```

- list 방식
- PATH 에서 자동 검색(리눅스가 알아서 /bin/ls 찾음)

```
char *args[] = {"ls", "-l", NULL};
execvp("ls", args);
```

- 배열 방식
- PATH 검색

```
char *env[] = {"MYVAR=hello", NULL};
execle("/bin/ls", "ls", "-l", NULL, env);
```

- list 방식
- 환경 변수 직접 설정 가능

```
char *args[] = {"ls", "-l", NULL};  
char *env[] = {"MYVAR=hello", NULL};  
  
execve("/bin/ls", args, env);
```

- 배열 방식
- 환경 변수 지정
- 가장 근본 함수

[01] exec() 시스템 콜과 프로세스 이미지 교체

[01-1] exec() 호출 후 실행 흐름은 어떻게 변하며 PID 는 유지되는가?

- 현재 실행 중인 코드를 버리고 새 프로그램을 로드했을 때, 프로세스의 정체성(PID)은 유지되는가?
- 호출 이후의 코드는 왜 실행되지 않는가?

[01-2] 시스템 레벨 관찰: 단일 프로세스 내 코드 영역 관찰하기

코드:

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("-- Hello -- (PID: %d)\n", getpid());

    execl("/bin/ls", "ls", "-l", NULL);

    printf("이 문구가 보이면 exec 실패\n");

    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
-- Hello -- (PID: 6944)
total 24
-rw-rw-r-- 1 pumpkinbee pumpkinbee 239 May  8 08:58 09_exec_basic.c
-rwxrwxr-x 1 pumpkinbee pumpkinbee 17536 May  8 08:58 09.out
```

strace 는 아래와 같다.

1. 기존 프로그램 실행: write(...) 호출 확인
2. 이미지 교체 단층점: execve(...) 시스템 콜 호출 성공(리턴값 0)
3. PID 유지 증명: execve 이후 자식 프로세스 생성이 없으며, set_tid_address(...)을 통해 동일한 PID 가 유지되고 있음을 확인
4. 새로운 자원 로드: execve 성공 직후, 기존의 라이브러리들을 버리고 ls 실행에 필요한 자원을 새롭게 mmap 하여 메모리에 적재
5. 기존 코드 소멸

결과 해석:

학습 결과 `execl()` 호출 이후에는 기존 프로그램의 마지막 `printf()` 문장이 실행되지 않았으며, 대신 `/bin/ls` 프로그램의 실행 결과가 출력되었다. 이는 `exec()` 계열 함수가 단순히 새로운 함수를 호출하는 방식이 아니라, 현재 실행 중인 프로세스의 실행 이미지를 새로운 프로그램으로 완전히 교체한다는 것을 의미한다.

표준 출력에서 먼저 “— Hello—” 문장이 출력된 이유는 `execl()` 호출 이전까지는 기존 프로그램의 코드가 정상적으로 실행되고 있었기 때문이다.

그러나 `execl("/bin/ls", "ls", "-l", NULL)`가 정상적으로 수행되는 순간, 현재 프로세스의 코드 영역, 데이터 영역, 힙, 스택이 모두 `/bin/ls` 프로그램의 메모리 구조로 교체된다.

그 결과 기존 프로그램의 아래코드:

```
printf("이 문구가 보이면 exec 실패Wn");
```

는 더 이상 메모리 상에 존재하지 않게 되므로 실행될 수 없다.

`strace` 결과에서도 이러한 동작을 확인할 수 있다.

```
execve("/bin/ls", ["ls", "-l"], ...) = 0
```

여기서 `exeve()`는 실제 커널 수준에서 수행되는 시스템 콜이며, 현재 프로세스 내부에 `/bin/ls` 실행 파일을 새롭게 적재(load) 하는 역할을 수행한다.

반환값이 0이라는 것은 시스템 콜이 성공적으로 수행되었음을 의미한다.

또한, 중요한 점은 프로그램은 변경되었지만 PID 는 유지된다는 것이다. `exec()`는 새로운 프로세스를 생성하는 함수가 아니라, 기존 프로세스 내부에서 실행 프로그램만 교체하는 메커니즘이다. 따라서 운영체제는 기존 PCB(task_struct)를 유지한 채, 메모리 공간만 새로운 프로그램으로 교체한다.

즉, `exec()` 이후의 프로세스는

- 동일한 PID 를 유지하지만
- 내부적으로는 완전히 다른 프로그램을 실행하게 되며
- 기존 코드 흐름은 완전히 소멸한다.

이를 통해 리눅스 운영체제는 프로세스 생성(fork)과 프로그램 실행(exec)을 분리하여 관리하고 있음을 확인할 수 있다.

[01-3] 이론

Process Image 재구성 및 PCB 재사용

exec() 시스템 콜은 새로운 프로세스를 생성하는 기능이 아니라, 현재 실행 중인 프로세스의 실행 이미지를 새로운 프로그램으로 교체하는 메커니즘이다. 따라서 exec()가 수행되면 프로그램의 메모리 구조는 제거되고, 새로운 실행 파일의 내용으로 다시 구성된다.

이 과정에서 프로세스의 메모리 영역은 다음과 같이 재구성된다.

- 코드 영역(Text Segment): 새 프로그램의 실행 코드로 교체
- 데이터 영역(Data Segment): 새 프로그램의 전역 변수 및 정적 변수로 초기화
- 힙 영역(Heap): 새 프로그램 기준으로 다시 생성
- 스택 영역(Stack): 새로운 함수 호출 구조와 지역 변수를 위한 공간으로 초기화

즉, 기존 프로그램이 사용하던 변수, 함수, 실행 흐름은 모두 제거되며, CPU 의 프로그램 카운터(PC) 역시 새 프로그램의 시작 지점을 가리키도록 변경된다. 결과적으로 프로세스 내부에서는 완전히 새로운 프로그램이 처음부터 실행되는 것과 같은 상태가 된다.

하지만 중요한 점은 프로세스 자체가 새로 생성되는 것은 아니라는 것이다. 운영체제는 기존 프로세스의 PCB(Process Control Block), 즉 리눅스의 task_struct 구조체를 그대로 유지한다.

따라서 다음과 같은 정보들은 exec() 이후에도 유지된다.

- PID(Process ID)
- PPID(Parent Process ID)
- 사용자 권한(UID/GID)
- 열린 파일 디스크립터(File Descriptor)
- 현재 작업 디렉터리(Current Working Directory)
- 프로세스 우선순위 및 일부 스케줄링 정보

즉, 운영체제 입장에서는 “같은 프로세스가 다른 프로그램으로 갈아탄 것”으로 처리된다. 프로세스의 정체성은 유지되지만, 내부 실행 내용만 완전히 바뀌는 것이다.

이러한 구조는 리눅스의 프로세스 관리 모델에서 매우 중요한 특징이다. 실제로 리눅스 셸은 새로운 명령어를 실행할 때 일반적으로

1. fork()로 자식 프로세스를 생성하고
2. 자식 프로세스 내부에서 exec()를 호출하여 원하는 프로그램으로 교체한다.

즉, fork() 는 “프로세스 생성”, exec()는 “프로그램 교체”라는 역할 분리를 기반으로 동작한다.

[01-4] 추가

Process Hollowing 과 exec() 메커니즘

(1) 프로세스 할로잉(Process Hollowing)의 원리

Process Hollowing 은 정상 프로그램처럼 보이는 프로세스를 생성한 뒤, 내부 실행 코드를 악성 코드로 바꿔치기하는 공격 기법이다.

공격 흐름은 일반적으로 다음과 같다.

- | |
|--|
| <ol style="list-style-type: none">1. 정상 프로그램(explorer.exe, notepad.exe 등)을 일시 중단 상태로 실행2. 정상 프로세스의 메모리 영역 제거(Unmap)3. 악성 코드를 해당 메모리 공간에 주입4. CPU 실행 흐름(EIP/RIP)을 악성 코드로 변경5. 프로세스 재개(Resume) |
|--|

겉으로 보면 PID 와 프로세스 이름은 정상 프로그램처럼 보이지만, 실제 메모리 내부에서는 악성 코드가 실행된다.

(2) 대응 방식

execve() 같은 시스템 콜을 중요한 감시 지점(Hook Point)으로 사용한다.

특히 다음 항목들을 검사한다.

- | |
|---|
| <ul style="list-style-type: none">- 실행 파일 해시(Hash)- 디지털 서명(Signature)- 부모-자식 프로세스 관계- 비정상 메모리 매핑- 코드 섹션 변조 여부 |
|---|

예를 들어:

- | |
|---|
| <ul style="list-style-type: none">- 정상 notepad.exe PID 인데 메모리 내부 코드가 알려진 정상 해시와 다르면 |
|---|

보안 솔루션은 이를 Process Hollowing 또는 코드 인젝션 공격으로 의심할 수 있다.

즉, 운영체제의 “프로세스 이미지 교체” 기능은 정상적인 프로그램 실행의 핵심이면서도, 동시에 공격자에게는 매우 강력한 은닉 기술의 기반이 되기도 한다.

[02] fork()와 exec()의 시너지: 셸(Shell)의 동작 원리

[02-1] 부모 프로세스를 유지하면서 새로운 프로그램을 실행하는 방법은?

[02-2] 시스템 레벨 관찰: 자식 프로세스의 변신과 부모의 대기 흐름 분석

코드:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("[부모] PID: %d\n", getpid());

    pid_t pid = fork();

    if(pid < 0){
        perror("fork 실패");
        return 0;
    }
    else if(pid == 0){
        printf("[자식] PID: %d\n", getpid());

        execl("/bin/date", "date", NULL);

        perror("exec 실패");
        return 1;
    }
    else {
        printf("[부모]a\n");
        wait(NULL);
        printf("[부모] - 자식 종료\n");
    }
    return 0;
}
```

결과:

표준 출력 결과는 아래와 같다.

```
[부모] PID: 5588
[부모]
[자식] PID: 5589
Fri May  8 12:49:17 PM UTC 2026
[부모] - 자식 종료
```

strace 결과 확인:

- clone() 시스템콜로 자식(PID 5589(생성))
- 부모(5588)는 wait4() 시스템 콜을 호출하며 블로킹(대기) 상태로 진입
- 자식(5589)은 execve("/bin/date",...)를 호출하여 자신의 이미지를 교체함

[02-3] 이론

(1) 리눅스 셸(Shell)의 실행 메커니즘

리눅스에서 사용자가 터미널에 명령어를 입력하면, 셸(Shell)은 단순히 프로그램을 실행하는 것이 아니라, “복제 후 교체(fork-exec 모델)”라는 구조를 통해 명령을 처리한다.

- fork()
 - 현재 실행 중인 셸 프로세스를 복제하여 새로운 자식 프로세스를 생성한다
 - 이 단계에서는 부모 셸과 동일한 코드와 메모리 상태를 가진 프로세스가 만들어진다
- exec()
 - 생성된 자식 프로세스의 실행 이미지를 새로운 프로그램으로 완전히 교체한다.
 - 예를 들어, ls 명령을 실행하면, 자식 프로세스는 ls 프로그램으로 변환된다

즉, fork 는 “공간 확보”, exec 는 “프로그램 교체”역할을 수행한다.

(2) Shell 의 동작 원리

사용자가 터미널에 다음과 같이 입력했다고 가정한다.

```
ls
```

셸 내부에서는 다음과 같은 과정이 발생한다.

1. 셸(bash)은 fork()를 호출하여 자신을 복제한다.
2. 복제된 자식 프로세스는 exec("ls") 호출하여 ls 프로그램으로 변경된다.
3. 부모 셸은 자식 프로세스가 종료될 때까지 wait() 상태로 대기한다.
4. 자식 프로세스가 종료되면 부모 셸은 다시 사용자 입력을 받을 준비 상태로 돌아간다.

이 구조는 시스템 안정성과 격리성을 크게 향상시킨다. 실행 중인 프로그램(자식)이 오류로 종료되더라도 부모 셸은 영향을 받지 않고 계속 동작한다. 각 프로그램 실행이 독립된 프로세스 공간에서 수행되며, 셸은 항상 안정적으로 사용자 입력을 받을 수 있다.

즉, 리눅스 셸은 “자식을 복제한 뒤, 그 복제본을 다른 프로그램으로 변환해서 실행시키는 구조”로 동작한다.

[02-4] 추가

(1) 악성코드의 프로세스 생성 감시

현대 보안 솔루션은 단순히 “어떤 프로그램이 실행되었는가?”만 보는 것이 아니라, “누가 누구를 실행했는가”까지 추적한다.

즉, 프로세스의 부모-자식 관계를 분석한다.

예를 들어,

- bash -> gcc : 정상 - bash -> ls : 정상 - notepad.exe -> cmd.exe : 비정상 가능성 높음 - PDF Reader -> powershell.exe : 악성 행위 의심
--

처럼, 일반 프로그램이 갑자기 셸(cmd, powershell)이나 네트워크 도구를 fork() + exec()로 실행하면 보안 엔진은 이를 이상 행위로 판단할 수 있다.

이는 악성 코드가 정상 프로그램 내부에서 몰래 명령 실행을 시도하는 경우가 많기 때문이다.

(2) 샌드박스(Sandbox) 기술의 기초

보안 제품은 의심 파일을 직접 실행하지 않고, 격리된 자식 프로세스 안에서만 실행한다.

동작 흐름은 다음과 같다.

- | |
|--|
| <ol style="list-style-type: none">1. 부모(보안 엔진)가 안전한 분석 환경 생성2. fork()로 자식 프로세스 생성3. 자식이 exec()로 의심 프로그램 실행4. 부모는 바깥에서 자식의 시스템 콜을 감시5. 위험 행동 발견 시 즉시 종료 |
|--|

예를 들어 다음 행위를 감시한다.

- 중요 파일 접근
- 네트워크 연결 시도
- 다른 프로세스 조작
- 권한 상승 시도
- /etc/passwd 수정
- execve() 연속 호출

즉, 운영체제의 프로세스 복제, 프로그램 교체, 부모-자식 구조 메커니즘 자체가 현대 보안 기술의 핵심 기반이 된다.

[03] execve()와 환경 변수: 보안 실행 문맥 설정

[03-1] 새로운 프로그램에 데이터와 환경 설정은 어떻게 전달되는가?

[03-2] 시스템 레벨 관찰: execve 인자(argv)와 환경변수(envp) 전달 과정

코드:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    char *argv[] = {"ls", "-al", NULL};

    // 사용자 정의 환경 변수 설정
    char *envp[] = {
        "MY_NAME=SU_USER",
        "CONF_LEVEL=HIGH",
        NULL
    };

    printf("[부모] 커스텀 환경 변수를 설정하고 ls 를 실행합니다.\n");

    if (execve("/bin/ls", argv, envp) == -1) {
        perror("execve 실패");
    }

    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
[부모] 커스텀 환경 변수를 설정하고 ls 를 실행합니다.
total 60
drwxrwxr-x 2 pumpkinbee pumpkinbee  4096 May  8 14:52 .
drwxrwxr-x 6 pumpkinbee pumpkinbee  4096 May  8 07:37 ..
-rw-rw-r-- 1 pumpkinbee pumpkinbee   239 May  8 08:58 01_exec_basic.c
-rwxrwxr-x 1 pumpkinbee pumpkinbee 17944 May  8 12:49 02.out
-rw-rw-r-- 1 pumpkinbee pumpkinbee   537 May  8 12:49 02_fork_exec.c
-rwxrwxr-x 1 pumpkinbee pumpkinbee 17480 May  8 14:52 03.out
-rw-rw-r-- 1 pumpkinbee pumpkinbee   428 May  8 14:52 03_execve_env.c
```

strace 실행 결과에서는 다음과 같은 핵심 시스템 콜이 관찰된다.

```
execve("/bin/ls", ["ls", "-al"], 0x7ffc1f2bda40 /* 2 vars */) = 0
```

결과 해석:

execve() 호출 시 커널은 기존 프로세스의 메모리를 새로운 프로그램(/bin/ls)으로 교체하면서, 함께 전달된 argv 와 envp 배열도 새 프로그램의 스택 영역에 저장한다.

strace 결과는 실행 인자와 환경 변수가 실제로 커널에 전달되었음을 보여준다.

또한, execve()는 리눅스 커널에 존재하는 실제 시스템 콜이며, execl(), execvp() 같은 함수들은 내부적으로 모두 execve()를 호출하는 Wrapper 함수이다.

즉, execve()는

- 프로그램 실행
- 실행 인자 전달
- 환경 변수 전달

을 동시에 수행하는 핵심 시스템 콜임을 확인할 수 있었다.

[03-3] 이론

유일한 진성 시스템 콜 execve()와 Wrapper 구조

리눅스 커널에서 실제로 존재하는 시스템 콜은 execve() 하나 뿐이다.

우리가 자주 사용하는

- execl()
- execvp()
- execv()
- execlp()

등의 함수들은 모두 glibc 라이브러리가 제공하는 Wrapper 함수이며, 내부적으로 최종적으로 execve() 시스템 콜을 호출한다.

즉, 사용자 프로그램은 라이브러리 함수를 사용하지만, 실제 커널 진입 지점은 execve() 하나로 통합되어 있다.

또한, execve()에 전달된 envp 배열은 새로운 프로그램의 환경 변수 공간에 저장된다.

이 데이터는 프로그램 내부의 전역 환경 변수 영역인 environ 에 유지되며, 프로그램은 어디서든 getenv() 함수를 통해 조회할 수 있다.

예를 들어

```
getenv("PATH");
```

와 같은 방식으로 환경 변수 값을 읽을 수 있다.

즉, execve()는 단순한 프로그램 실행뿐 아니라, 새로운 프로그램의 실행 문맥까지 함께 구성하는 역할을 수행한다.

[03-4] 추가

환경 변수 조작 공격과 방어(LD_PRELOAD)

- 위험성

LD_PRELOAD 는 리눅스 동적 링커가 지정된 라이브러리를 먼저 로드하도록 만드는 환경 변수이다.

공격자는 이를 이용해 open(), read(), execve() 같은 시스템 콜 함수를 가로채어 동작을 변조할 수 있다.

예를 들어, 정상 프로그램 실행 전에 악성 라이브러리를 삽입하여 파일 접근을 감시하거나 인증 로직을 위화하는 공격이 가능하다.

- execve()와의 연관성

execve()는 새로운 프로그램을 실행할 때 envp 배열을 함께 전달한다.

따라서 부모 프로세스가 오염된 환경 변수(LD_PRELOAD=malicious.so)를 넘기면, 새 프로그램은 시작과 동시에 공격자 코드가 삽입된 상태로 실행될 수 있다.

- 보안 전략:

보안 솔루션과 시스템 프로그램은 외부 프로그램 실행 전 환경 변수 정화(Sanitization)를 수행한다.

특히, LD_PRELOAD, LD_LIBRARY_PATH 같은 위험 변수를 제거하거나 무시하여, 신뢰되지 않은 라이브러리 로딩을 차단한다.

- 실무 연관성

sudo, sshd, 브라우저 샌드 박스, 백신 엔진 등은 실행 시 위험 환경 변수를 자동 제거하여 권한 상승 및 코드 삽입 공격을 방어한다.

[04] 자원 상속의 그림자: exec() 이후 파일 디스크립터(FD) 생존

[04-1] 프로세스 이미지가 바뀌어도 부모가 열어둔 파일 권한이 상속되는가?

[04-2] 시스템 레벨 관찰: exec() 이후에도 닫히지 않은 FD 를 통한 무단 접근 증명 코드:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    // 1. 부모가 비밀 파일을 연다.
    int fd = open("secret.txt", O_RDWR | O_CREAT, 0644);
    write(fd, "TOP_SECRET_DATA\n", 16);

    printf("[부모] 비밀 파일을 열었습니다 (FD 번호: %d). 이제 cat 으로 변신합니다.\n",
fd);

    // 2. /bin/cat 으로 변신하면서, fd 번호를 인자로 넘겨준다.
    char fd_path[50];
    sprintf(fd_path, "/dev/fd/%d", fd);
    execl("/bin/cat", "cat", fd_path, NULL);

    return 0;
}
```

결과:

표준 출력은 아래와 같다.

```
[부모] - 비밀 파일 열기(FD: 3)
[부모] - cat 으로 변신
TOP_SECRET_DATA
```

strace 결과는 아래와 같다.

```
openat(AT_FDCWD, "/dev/fd/3", O_RDONLY) = 4
```


결과 해석:

학습 결과 `exec()`는 프로세스의 메모리 이미지(코드, 데이터, 스택)만 새로운 프로그램으로 교체할 뿐, 기존에 열려 있던 파일 디스크립터(File Descriptor)는 유지된다는 사실을 확인할 수 있었다.

부모 프로세스가 `secret.txt` 를 열어 FD 3 번을 확보한 뒤 `cat` 프로그램으로 변신했지만, `cat` 은 해당 FD 를 그대로 상속받아 파일 내용을 출력할 수 있었다.

`strace` 결과를 보면 `cat` 프로그램이 부모에게서 상속받은 FD 3 번을 통해 파일에 접근했음을 보여준다.

즉, `exec` 이후에도 프로세스가 보유한 커널 자원(File Descriptor Table)은 유지되며, 새 프로그램 역시 동일한 자원에 접근할 수 있음을 확인할 수 있었다.

[04-3] 이론

(1) FD Table 의 지속성과 커널 자원 유지

`exec()` 시스템 콜은 프로세스의 메모리 영역만 새로운 프로그램으로 교체하며, 프로세스가 소유한 파일 디스크립터(FD Table)은 그대로 유지한다.

따라서, `exec` 이후 실행되는 새 프로그램도 이전 프로세스가 열어둔 파일, 파이프, 소켓 등에 계속 접근할 수 있다.

또한 커널 내부의 Open File Description 은 참조 계수(Reference Count)를 사용해 관리되므로, 프로세스 이미지가 변경되어도 참조가 남아있는 동안 파일은 닫히지 않는다.

이러한 구조 때문에

- 셸의 파이프
- 입출력 리다이렉션
- 서버 소켓 상속

등이 가능해진다.

반대로 민감한 FD 를 닫지 않고, `exec()`를 수행하면, 새 프로그램이 기존 권한을 그대로 사용할 수 있으므로 보안 취약점의 원인이 될 수도 있다.

[04-4] 추가

(1) 권한 유출 방어를 위한 FD_CLOEXEC

exec()는 프로세스의 메모리만 교체할 뿐, 기본적으로 파일 디스크립터(FD)는 유지한다. 따라서 관리자 권한(root)으로 열린 파일이나 소켓이 일반 프로그램으로 그대로 상속되면 심각한 보안 취약점이 발생할 수 있다.

예를 들어,

- 보안 프로그램이 root 권한으로 민감한 파일을 열어둔 상태에서
- exec()로 일반 사용자 프로그램을 실행하면
- 새 프로그램이 해당 FD를 통해 관리자 권한 자원에 접근할 수 있게 된다.

이를 방지하기 위해 리눅스는 FD_CLOEXEC(Close-on-Exec)메커니즘을 제공한다.

(2) 방어 방법:

파일을 열 때

```
open("secret.txt", O_RDONLY | O_CLOEXEC);
```

처럼 O_CLOEXEC 플래그를 사용하거나,

이미 열린 FD에 대해,

```
fcntl(fd, F_SETFD, FD_CLOEXEC);
```

를 설정할 수 있다.

(3) 동작 원리

FD_CLOEXEC가 설정된 파일 디스크립터는:

- exec() 호출이 성공하는 순간
- 커널이 자동으로 해당 FD를 닫는다.

즉, 새 프로그램에는 해당 자원이 전달되지 않는다.

이 메커니즘은

- 권한 있는 파일 핸들 유출 방지
- 인증 소켓 탈취 차단
- 민감한 IPC 자원 보호
- Privilege Escalation 방어

에 매우 중요하다.

실제 운영체제와 보안 솔루션은 민감한 FD에 기본적으로 CLOEXEC를 설정하여, exec 이후 자원 상속이 발생하지 않도록 관리한다.